Colegio de San Gabriel Arcangel, Inc.
Founded 1993
City of San Jose del Monte, Bulacan, Philippines

COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION

A Capstone Project
Submitted to the Faculty of the College of Computer Studies
Colegio de San Gabriel Arcangel, Inc.

In Partial Fulfillment of the Requirements for the
Degree of Bachelor of Science in Information Technology

Submitted by:

Mark Danzen Q. Aquino
Darell F. Bacani
Christian Gold DC. Española
Markgil L. Gamboa
Irish I. Gregorio
Nicole S. Mijoy
Emmanuel A. Spicer

Prof. Jimmy D.V. Roldan
Research Group's Adviser

Submitted to:

Jimmy D.V. Roldan, MSIT
Research Course Facilitator

October 2026

CERTIFICATE OF ORIGINALITY

This is to certify that the intellectual content of this research paper entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION"** is the result of our collective ideas derived from the data gathered with the guidance of the professionals who have assisted us in the construction of this research. Even though we received help with style, presentation, and language expression from others, we utilized suitable citations in referring to other works from other sources.

This research paper does not contain any previously published or written material by another person which, to a substantial extent, has been accepted for any other degree or diploma at Colegio de San Gabriel Arcangel, Inc. or any other educational institution, except where due acknowledgment is made in the paper. Any contributions made to the research by others, with whom we have collaborated at Colegio de San Gabriel Arcangel, Inc. or elsewhere, are explicitly acknowledged.

We confirm that we understand plagiarism and accept the repercussions of any plagiarized work submitted by us, either individually or as part of a group.

Researchers:

Mark Danzen Q. Aquino	**Darell F. Bacani**
Christian Gold DC. Española	**Markgil L. Gamboa**
Irish I. Gregorio	**Nicole S. Mijoy**
Emmanuel A. Spicer	

Attested by:

Prof. Jimmy D.V. Roldan	**Jimmy D.V. Roldan, MSIT**
Research Group's Adviser	Research Course Facilitator

Date: _____

CERTIFICATE OF EDITING

This is to certify that I, the grammarian, have edited this thesis entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION"**

prepared by:

Mark Danzen Q. Aquino
Darell F. Bacani
Christian Gold DC. Española
Markgil L. Gamboa
Irish I. Gregorio
Nicole S. Mijoy
Emmanuel A. Spicer

and have found it to be thorough and acceptable with respect to grammar and composition.

Name of Grammarian
Signature over printed name
Designation: _____
Date of Completion: _____

APPROVAL SHEET

This capstone project entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION"** was prepared and submitted by:

Mark Danzen Q. Aquino
Christian Gold DC. Española
Irish I. Gregorio
Emmanuel A. Spicer
Darell F. Bacani
Markgil L. Gamboa
Nicole S. Mijoy

in partial fulfillment of the requirements for the degree of **Bachelor of Science in Information Technology** has been examined and hereby recommended for approval and acceptance for ORAL EXAMINATION.

Prof. Jimmy D.V. Roldan	**Jimmy D.V. Roldan, MSIT**
Research Group's Adviser	Research Course Facilitator

APPROVED by the COMMITTEE ON ORAL EXAMINATION with a grade of _____.

Name of Panel Member	**Name of Panel Member**
Member	Member

Name of Panel Chairman
Chairman

Date of Oral Examination: _____

ACCEPTED and APPROVED as partial fulfillment of the requirements for the degree of **Bachelor of Science in Information Technology**.

Name of Dean
Dean, College of Computer Studies

ACKNOWLEDGMENTS

[Draft — please personalize.] The researchers wish to express their sincere gratitude to the people who made this capstone project possible.

To **Prof. Jimmy D.V. Roldan**, our Research Group's Adviser and Research Course Facilitator, for his patience, guidance, and constructive feedback throughout the development of COBRA and the writing of this paper.

To the members of the panel, for their valuable comments and suggestions that helped improve the system and this study.

To the respondents and the expert evaluators, who generously gave their time to use and evaluate the system.

To our families and friends, for their unending support, understanding, and encouragement.

Above all, to Almighty God, for the wisdom, strength, and perseverance that carried us through this project.

— The Researchers

ABSTRACT

Many beginners find programming difficult to learn on their own. Existing online platforms for Python often lock practice and support behind paid subscriptions, give limited feedback, and treat every learner the same way. This capstone project developed **COBRA: Comprehensive Online Beginner Resource for Python Application**, a web-based learning system for learners with little to no programming experience. The system was built using the Agile methodology with Python (Flask) on the server, HTML, CSS, and JavaScript on the client, and a MariaDB/MySQL database. Python code runs directly in the learner's browser through Pyodide, so learners can practice without installing anything.

COBRA is made up of eleven modules: security, user registration, lesson management, learning resources, coding exercises, learning activities, coding sandbox, progress monitoring, learning analytics, lesson recommendation, and achievement tracking. Lessons are organized into chapters and modules that unlock in order once a learner reaches the 80% pass mark. Each lesson combines reading content and a video tutorial with three game-based activities (Multiple Choice, Fill in the Blanks, and Flashcards) and an output-graded coding exercise that gives immediate feedback. Missed items are traced back to the part of the lesson that teaches them and recommended for review, while badges, a learning map, and a certificate of completion keep learners motivated. Mentors manage and publish content, and administrators monitor progress, analytics, and system activity.

The system was evaluated by [number] respondents using a survey questionnaire based on the ISO 9126 software quality characteristics of functionality, reliability, usability, and efficiency. [Insert the overall weighted mean and its verbal interpretation once the evaluation is complete.]

Keywords: Python programming, beginner learners, web-based learning system, interactive learning, learning analytics, lesson recommendation, gamification, ISO 9126

TABLE OF CONTENTS

		PAGE
Title Page		1
Certificate of Originality		2
Certificate of Editing		3
Approval Sheet		4
Acknowledgments		5
Abstract		6
Table of Contents		7
List of Tables		10
List of Figures		11
Chapter 1:	**INTRODUCTION, REVIEW OF RELATED LITERATURE, AND THE PROBLEM**	1
	Introduction	1
	Review of Related Literature	5
	Theoretical Framework	8
	Conceptual Framework	9
	Statement of the Problem	10
	Hypothesis	13
	Scope and Delimitation of the Study	13
	Significance of the Study	15
	Definition of Terms	16
Chapter 2:	**METHODOLOGY**	19
	Research Design	19
	Research Locale	19
	Respondents of the Study	20
	Sampling Design	20
	Instrumentation	21
	Validation of the Instrument	21
	Data Gathering Procedure	21
	Ethical Considerations	22
	Statistical Treatment of the Data	23
	Software Development Methodology	23
	Calendar of Activities	26
	Resources	27

Chapter 3:	**RESULT, ANALYSIS, AND DISCUSSION**	31
	Security Module	31
	User Registration Module	32
	Lesson Management Module	33
	Learning Resources Module	34
	Coding Exercises Module	34
	Learning Activities Module	35
	Coding Sandbox Module	36
	Progress Monitoring Module	37
	Learning Analytics Module	37
	Lesson Recommendation Module	38
	Achievement Tracking Module	39
	Evaluation of the System Based on ISO 9126	40
	System Architecture	42
Chapter 4:	**SUMMARY OF FINDINGS, CONCLUSIONS, AND RECOMMENDATIONS**	44
	Summary of Findings	44
	Conclusions	45
	Recommendations	47
	Other Recommendations	47
References		49
Appendices		52
	A – Letter to Respondents	52
	B – Letter of Permission to the Dean	53
	C – Letter to Expert Validators of the Survey Questionnaire	54
	D – Research Instrument Validation Sheet	55
	E – Survey Questionnaire	57
	F – Formula with Sample Computation of Statistics Used	59
	G – Raw Data	60
	H – System Screenshots	61
Curriculum Vitae		62

LIST OF TABLES

TABLE NO.	TITLE	PAGE
1	Comparison of Existing Python Learning Platforms and COBRA	4
2	Distribution of Respondents	20
3	Likert Scale and Verbal Interpretation	21
4	Description of Activities	26
5	Gantt Chart of Activities	27
6	Hardware Requirements	27
7	Software Requirements	28
8	Learning Activity Types and the Skills They Measure	35
9	Starter Badges of COBRA	39
10	Respondents' Evaluation of COBRA in Terms of Functionality	40
11	Respondents' Evaluation of COBRA in Terms of Reliability	41
12	Respondents' Evaluation of COBRA in Terms of Usability	41
13	Respondents' Evaluation of COBRA in Terms of Efficiency	41
14	Summary of the Respondents' Evaluation of COBRA	42

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
1	Theoretical Framework of the Study	9
2	Conceptual Framework of the Study	10
3	Agile Methodology	24
4	System Architecture of COBRA	43

Chapter 1

INTRODUCTION, REVIEW OF RELATED LITERATURE, AND THE PROBLEM

This chapter presents the introduction and background of the study, the review of related literature, the theoretical and conceptual frameworks, the statement of the problem and the objectives of the study, the hypothesis, the scope and delimitation, the significance of the study, and the definition of terms used throughout the paper.

Introduction

Programming is a popular, powerful, and important skill in technology-related careers, yet many beginners have a hard time learning it effectively. Python has become one of the most widely used programming languages in the world, in large part because its simple and readable syntax makes it friendly to beginners.

There are a number of online learning platforms that offer resources for learning Python. However, some of these platforms provide only limited feedback, lack interactive and engaging features that encourage learners to keep going, and offer little progress tracking that could identify a learner's strengths and weaknesses based on their actual performance.

In the technology industry, programming is one of the most important skills a person can have. Helping beginners build a strong foundation prepares them for

future career opportunities, while a weak foundation can hold them back once topics become more difficult.

To address this, the researchers propose **COBRA**, a Comprehensive Online Beginner Resource for Python Application. Through structured lessons, video tutorials, game-based activities, coding exercises with immediate feedback, and an interactive practice environment, the system aims to give beginners the support they need to stay motivated and engaged while improving their programming skills.

Background of the Problem

Learning programming benefits those who are in education and in technology-related careers. It is often described as easy to start but hard to master. According to Oka et al. (2024), novice programmers commonly lack problem-solving skills: they have difficulty planning abstract solutions and expressing them as a program, which causes them to get stuck when they meet difficult programming concepts.

Many learners use several online platforms to study programming. Although there are many such platforms on the internet, learners are often unsure of what they are supposed to study and in what order. Chen et al. (2024) emphasize that insufficient guidance is one of the struggles that slows down a beginner's learning progress. Alasmari et al. (2024) similarly found that several popular online coding tutorial systems lack features that address the difficulties novice programmers actually experience.

Learning the fundamental concepts of programming builds a strong foundation. Uhanova et al. (2023) found, in a study of 1,744 introductory programming students, that students who took an additional course on the basics performed better academically. When beginners skip the fundamentals, they tend to get stuck repeatedly once topics become harder, which can affect their ability to develop their skills and their motivation to pursue a technology-related career.

Python is considered a multi-purpose programming language. It is beginner-friendly because its simple yet powerful syntax makes many tasks straightforward. GitHub (2024) reported that in 2024 Python overtook JavaScript as the most-used programming language on GitHub, which increases the number of potential beginners who want to learn it.

Using Python as the language of instruction, this study offers a solution that helps beginners build their foundation and improve their learning through interactive practice.

Overview of the Current State of the Technology

Learning a programming language, specifically Python, usually begins with a beginner searching the internet for an online platform to learn from. Numerous platforms offer Python learning resources, each with a different approach and set of features.

W3Schools, Mimo, Learn Python, and SoloLearn are some of the well-known platforms a beginner will find. W3Schools is a web-based learning platform that provides tutorials, examples, quizzes, exercises, and a coding practice

environment. Mimo is a mobile and web learning platform that uses short lessons and exercises with gamification and progress tracking. Learn Python is a mobile learning application that provides structured lessons, quizzes, exercises, and community discussions. SoloLearn offers bite-sized lessons, a code playground, and a community of learners.

Despite the effectiveness of these platforms, some limitations remain. First, some platforms require a paid subscription for additional learning resources, practice, and support. Second, platforms differ in how they handle learners' different levels of knowledge and skill, and few of them point a learner back to the specific part of a lesson they need to review. Table 1 compares these platforms with the proposed system.

Table 1
Comparison of Existing Python Learning Platforms and COBRA

Feature	W3Schools	Mimo	Learn Python	SoloLearn	COBRA
Structured beginner lessons	✓	✓	✓	✓	✓
Video tutorials	–	–	–	–	✓
Quizzes / game-based activities	✓	✓	✓	✓	✓
Coding exercises with immediate feedback	✓	✓	✓	✓	✓
Browser-based coding sandbox	✓	verify	verify	✓	✓
Progress tracking	Paid	✓	✓	✓	✓
Strengths and weaknesses per lesson	–	–	–	–	✓
Recommendation of the lesson part to review	–	–	–	–	✓
Badges / achievements	–	✓	verify	✓	✓
Mentor-managed content and learner monitoring	–	–	–	–	✓
Free to use	Partly	Partly	Partly	Partly	✓

Note: The comparison reflects the researchers' observation of each platform's free version; please re-check each cell against the current version of the platforms before submission.

Existing online programming platforms already offer interactive features, structured lessons, exercises, quizzes, coding environments, and progress tracking. However, each handles beginners differently. COBRA addresses the limitations identified above by combining structured lessons, video tutorials, interactive practice, and automatic recommendations that fit each beginner's needs, free of charge.

Project Rationale

COBRA was developed because beginners need more than a list of tutorials: they need a guided path, practice that responds to their mistakes, and a clear picture of what they still need to work on. By keeping lessons short and ordered, grading exercises and activities immediately, and pointing each learner back to the exact lesson part behind a missed item, COBRA aims to keep beginners from getting stuck and giving up. At the same time, mentors and administrators gain the tools to manage the course and monitor learners who need help.

Review of Related Literature

This section presents literature and studies that guided the design of COBRA.

Difficulties of Novice Programmers

Oka et al. (2024) identified the lack of problem-solving skills as one of the main issues faced by novice programmers, who have difficulty planning abstract solutions and expressing them in a program. Their choice-based learning method

let students focus on solution ideas without being slowed down by coding errors. Alasmari et al. (2024) reviewed the difficulties novices face and examined whether popular online coding tutorial systems address them; they found that several of these systems lack features that could effectively help novice programmers and called for more learner-centered environments. Chen et al. (2024) likewise point to insufficient guidance as one of the struggles of beginners.

Importance of a Strong Foundation

Uhanova et al. (2023) examined the link between first-year students' prior programming knowledge and their performance in an introductory programming course. Based on a sample of 1,744 students, those who attended an additional course on algorithmization performed better, showing the value of a solid foundation in the basics. This supports COBRA's design of requiring learners to pass each module before moving on to the next.

Structured and Manageable Content

Skulmowski and Xu (2022) discuss how the design of digital and online learning environments affects cognitive load. Interactive media, immersion, and redundant elements add load that is not related to the task, so designers should weigh whether such elements serve the intended learning outcome. Online learning environments should therefore focus on the resources learners actually need and present information in manageable portions. Menon (2023) adds that giving learners step-by-step, structured information helps them better understand the fundamental concepts of programming.

Practice and Immediate Feedback

Iftikhar (2022) notes that online programming activities let beginners practice and develop their problem-solving skills and understand programming concepts better. Marwan et al. (2022) designed and evaluated an adaptive immediate feedback system for block-based programming that gives students real-time information about their progress, correctness, and errors. The system improved students' performance and increased the number of students who completed their programming assignments. These findings support COBRA's exercises and activities, which grade every answer as soon as it is submitted.

Personalized Learning

Wu et al. (2024) describe personalized learning as an approach that identifies what each learner needs based on their strengths and weaknesses and uses this to improve their learning experience and outcomes. COBRA applies this idea by computing each learner's strengths and weaknesses per lesson from their first-try results and by recommending the specific lesson part that teaches each item they missed.

Synthesis

The reviewed literature shows that beginners struggle with problem solving and with knowing what to study, that a strong grasp of the fundamentals improves later performance, that content should be structured and kept manageable, and that immediate feedback and personalization improve both performance and persistence. Existing platforms cover some of these needs, but not all of them

together and not always for free. COBRA brings these elements into one web-based system for beginner Python learners.

Theoretical Framework

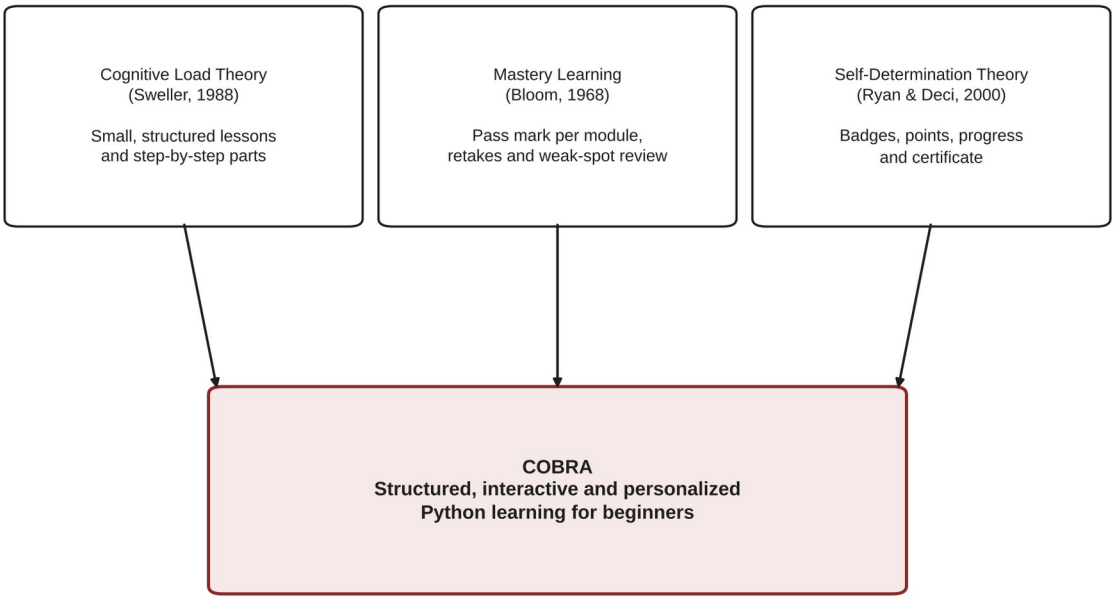
The study is anchored on three learning theories, shown in Figure 1.

Cognitive Load Theory. Sweller (1988) proposed that working memory is limited, so instruction should avoid overloading learners with information that does not contribute to learning. COBRA applies this by dividing the course into chapters, modules, and short lessons, splitting lesson content into headed parts, and showing only five questions per activity play.

Mastery Learning. Bloom (1968) argued that most learners can master a subject if they are given enough time, feedback, and corrective help before moving on. COBRA applies this through an 80% module pass mark, retakes of missed items, and a Module Review that must be completed before the next module unlocks.

Self-Determination Theory. Ryan and Deci (2000) explain that motivation grows when people feel competent, autonomous, and related to others. COBRA supports competence and motivation through points, badges, a visual learning map, and a certificate of completion, and supports autonomy through the free coding sandbox and optional review of passed modules.

Figure 1
Theoretical Framework of the Study

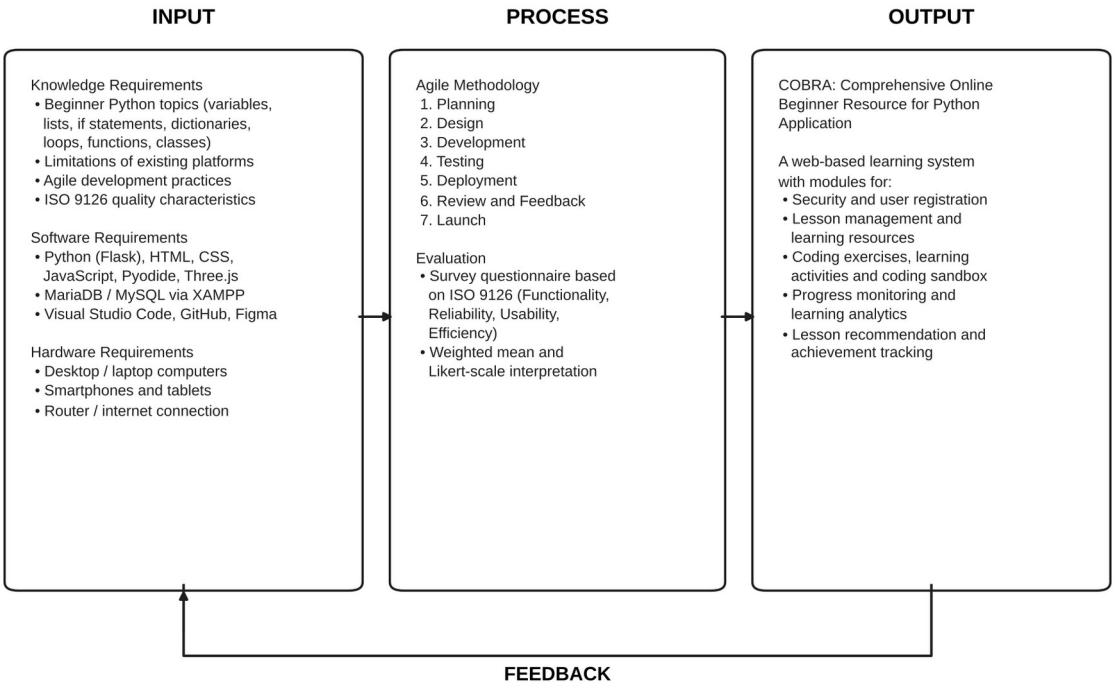


Conceptual Framework

The study uses the Input-Process-Output (IPO) model shown in Figure 2. The **input** consists of the knowledge, software, and hardware requirements of the system. The **process** follows the phases of the Agile methodology, followed by the evaluation of the system using a survey questionnaire based on ISO 9126. The **output** is COBRA: Comprehensive Online Beginner Resource for Python Application. Feedback from the evaluation is used to improve the system.

Figure 2

Conceptual Framework of the Study



Statement of the Problem

The general problem of this study is how to develop “COBRA,” a Comprehensive Online Beginner Resource for Python Application.

Specific Problems

1. How to create a module for security?
2. How to create a module for user registration?
3. How to create a module for lesson management?
4. How to create a module for learning resources?
5. How to create a module for coding exercises?
6. How to create a module for learning activities?

7. How to create a module for a coding sandbox?
8. How to create a module for progress monitoring?
9. How to create a module for learning analytics?
10. How to create a module for lesson recommendation?
11. How to create a module for achievement tracking?
12. How do the respondents evaluate the COBRA system with respect to the ISO 9126 metrics on: (a) functionality; (b) reliability; (c) usability; and (d) efficiency?
13. What system architecture was proposed for the COBRA system?

General Objective

To develop “COBRA,” a Comprehensive Online Beginner Resource for Python Application.

Specific Objectives

1. To develop a module for security. This ensures that only authorized users can access protected system resources through secure authentication, role-based access, and account management.
2. To develop a module for user registration. This allows new users to create learner accounts and access the learning features of the system.
3. To develop a module for lesson management. This offers structured Python programming lessons for beginners, organized into chapters, modules, and lessons.

4. To develop a module for learning resources. This provides lesson content, instructional videos, and supplementary materials that support each lesson topic.
5. To develop a module for coding exercises. This allows learners to practice Python by solving programming challenges that are run and graded immediately.
6. To develop a module for learning activities. This evaluates learners' understanding through game-based activities and gives immediate feedback on their answers.
7. To develop a module for a coding sandbox. This allows learners to write, run, and save Python programs in a safe web-based environment.
8. To develop a module for progress monitoring. This tracks learners' lesson completion and performance throughout the learning process.
9. To develop a module for learning analytics. This identifies learners' strengths and weaknesses based on their performance in activities and exercises.
10. To develop a module for lesson recommendation. This suggests the lessons and lesson parts each learner should review based on their identified weak areas.
11. To develop a module for achievement tracking. This awards badges and a certificate of completion to encourage engagement and motivation by recognizing learners' effort and completed goals.

12. To evaluate the COBRA system with respect to the ISO 9126 characteristics of functionality, reliability, usability, and efficiency.
13. To design the system architecture of the COBRA system.

Hypothesis

This study is developmental and descriptive in nature: it builds a system and describes how respondents evaluate it, so no statistical hypothesis is tested. The researchers expect that the respondents will evaluate COBRA as at least “Agree” (a weighted mean of 3.41 or higher) in each of the ISO 9126 characteristics of functionality, reliability, usability, and efficiency.

Scope and Delimitation of the Study

Scope

The study covers the development and evaluation of COBRA, a web-based learning system for beginner Python learners with three types of users: learners, mentors, and administrators. It includes the following modules:

1. **Security** – learner and staff sign-in, password hashing and rules, account lockout after five failed attempts, one-time codes sent by email, role-based access, automatic logout after inactivity, and activity and login logs.
2. **User Registration** – learner sign-up with email verification, age checking, and acceptance of the Terms and Conditions and Privacy

Notice; creation of mentor and administrator accounts by an administrator.

3. **Lesson Management** – creating, editing, ordering, archiving, previewing, and publishing chapters, modules, and lessons.
4. **Learning Resources** – lesson content with runnable code examples and embedded video tutorials.
5. **Coding Exercises** – programming challenges graded on their output and required code elements.
6. **Learning Activities** – Multiple Choice, Fill in the Blanks, and Flashcards activities with lives, timers, immediate feedback, and retakes.
7. **Coding Sandbox** – a browser-based Python environment where learners can write, run, and save code.
8. **Progress Monitoring** – lesson, module, and chapter progress for learners and staff.
9. **Learning Analytics** – dashboards, per-lesson strengths and weaknesses, and performance reports.
10. **Lesson Recommendation** – recommendations of the lesson part to review for every missed item.
11. **Achievement Tracking** – badges, notifications, and a certificate of completion.

Delimitation

1. The system is limited to basic Python programming lessons only.

2. The system is intended for beginner learners with little to no programming experience.
3. The system requires an internet connection for access and use.
4. The system is accessible only through web browsers; it has no native mobile application.
5. The system does not have SMS or offline notification features; notifications are shown inside the system and one-time codes are sent by email.
6. Coding exercises and the sandbox run Python in the browser through Pyodide, so programs that need files on the learner's computer, network access, or packages not available in Pyodide are not supported.
7. The evaluation is limited to the respondents described in Chapter 2 and to the ISO 9126 characteristics of functionality, reliability, usability, and efficiency.

Significance of the Study

The COBRA system will benefit the following:

Students. Students taking programming courses can practice coding and reinforce what they learn in class.

Self-learners. Self-learners can access learning materials anytime and anywhere.

Career changers. Career changers can learn and practice Python programming, which helps them develop their skills and prepare for technology careers.

Instructors. Instructors can recommend COBRA as a practice and skill-development tool for Python programming and, as mentors, monitor learners who need help.

Schools and educational institutions. Schools can use COBRA to support their programming education.

Researchers. The researchers enhance their knowledge and skills in software development, systems analysis, web technologies, and educational technology research.

Future researchers. The study can serve as a reference for future researchers and developers working on web-based learning systems, programming education, and adaptive learning technologies.

Definition of Terms

The following terms are defined as they are used in this study.

Achievement Tracking – The module that awards badges and a certificate of completion when a learner meets set requirements.

Administrator (Admin) – A staff user who manages accounts and monitors progress, analytics, reports, logs, and system health.

Agile Methodology – An iterative software development approach in which the system is planned, designed, built, tested, and improved in repeated cycles based on feedback.

Badge – A digital award shown on the learner's profile when a requirement, such as completing ten lessons, is met.

Chapter – The highest level of the course structure; it contains modules (e.g., "Introducing Lists").

Coding Exercise – A programming challenge at the end of a lesson that is graded by running the learner's code and comparing its output with the expected output.

Coding Sandbox – A web-based environment where learners can freely write, run, and save Python code.

Flask – A lightweight Python web framework used to build the server side of COBRA.

ISO 9126 – An international standard that defines a model for evaluating software product quality.

Learner – A user who studies the lessons and takes the activities and exercises in COBRA.

Learning Activity – A game-based assessment (Multiple Choice, Fill in the Blanks, or Flashcards) attached to a lesson.

Learning Analytics – The collection and analysis of learner data to understand and improve learning.

Lesson – A unit of instruction inside a module consisting of content, an optional video, activities, and an optional coding exercise.

Mentor – A staff user who creates, edits, and publishes course content and badges.

Module – A group of related lessons inside a chapter; the next module unlocks once the learner passes the current one.

Pass Mark – The minimum module average (80%) a learner must reach to unlock the next module.

Pyodide – A version of Python that runs inside the web browser, used by COBRA to run learners' code.

Weak Spot – An item the learner missed on the first try, linked to the part of the lesson that teaches it.

Chapter 2

METHODOLOGY

This chapter presents the research design, research locale, respondents, sampling design, instrumentation, validation of the instrument, data gathering procedure, ethical considerations, and statistical treatment of the data. It also describes the software development methodology, the calendar of activities, and the resources used to develop COBRA.

Research Design

The study uses the **developmental research design**, which involves the systematic design, development, and evaluation of a product, together with the **descriptive research design**, which describes the respondents' evaluation of the developed system. The researchers developed COBRA using the Agile methodology and then gathered quantitative data through a survey questionnaire to describe how the respondents rate the system in terms of the ISO 9126 quality characteristics.

Research Locale

The study is conducted at **Colegio de San Gabriel Arcangel, Inc.**, located in the City of San Jose del Monte, Bulacan, Philippines. The school offers the Bachelor of Science in Information Technology program, whose students and

faculty are the intended respondents of the study. Because COBRA is web-based, respondents may also access and evaluate the system online.

Respondents of the Study

The respondents are grouped into three categories: (1) **beginner learners** – students with little to no Python programming experience who use COBRA as learners; (2) **IT instructors** – faculty members who use COBRA as mentors and evaluate its content-management and monitoring features; and (3) **IT experts** – professionals or faculty with experience in software development who evaluate the technical quality of the system. Table 2 shows the distribution of respondents.

Table 2
Distribution of Respondents

Respondents	Number	Percentage
Beginner learners	30	75%
IT instructors	5	12.5%
IT experts	5	12.5%
Total	40	**100%**

Note: The numbers above are proposed; replace them with the actual number of respondents.

Sampling Design

The study uses **purposive sampling**, a non-probability sampling technique in which respondents are chosen because they meet criteria set by the researchers. Learner respondents must have little to no prior Python programming experience; instructor respondents must be teaching or have taught programming; and expert respondents must have experience in software development or quality assurance.

Instrumentation

The main instrument is a researcher-made **survey questionnaire** based on the ISO 9126 software quality model (see Appendix E). It has two parts: Part I gathers the respondent's profile (respondent type and prior programming experience), and Part II contains statements for each of the four characteristics evaluated – functionality, reliability, usability, and efficiency. Each statement is rated on a five-point Likert scale (Likert, 1932), shown in Table 3.

Table 3
Likert Scale and Verbal Interpretation

Scale	Range	Verbal Interpretation
5	4.21 – 5.00	Strongly Agree
4	3.41 – 4.20	Agree
3	2.61 – 3.40	Neutral
2	1.81 – 2.60	Disagree
1	1.00 – 1.80	Strongly Disagree

Validation of the Instrument

Before it is administered, the questionnaire is validated by three (3) expert validators in the field of information technology using the Research Instrument Validation Sheet (Appendix D), adapted from Samosa (2021). The validators rate the clarity, presentation and organization, suitability, adequateness, attainment of purpose, objectivity, and rating scale of the items. Their comments and suggestions are used to revise the questionnaire before it is distributed.

Data Gathering Procedure

The researchers follow these steps in gathering data:

1. Secure permission from the Dean of the college to conduct the study and evaluation (Appendix B).
2. Validate the survey questionnaire with the expert validators (Appendices C and D).
3. Orient the respondents about the purpose of the study and give them access to COBRA with test accounts.
4. Let the respondents use the system: learners go through the lessons, activities, exercises, and sandbox, while instructors and experts also use the mentor and administrator features.
5. Distribute the survey questionnaire through Google Forms (Appendices A and E).
6. Collect, tally, and analyze the responses using the statistical treatment described below.

Ethical Considerations

The researchers observe the following ethical standards. Participation is voluntary, and respondents may withdraw at any time without penalty. Informed consent is obtained before the evaluation. Following the Data Privacy Act of 2012 (Republic Act No. 10173), the personal information of respondents is kept confidential and used only for this study. Within the system itself, learners must accept the Terms and Conditions and Privacy Notice before using COBRA, learners under 18 must declare that a parent or guardian agrees, passwords are

stored only as hashes, and learners may request the deletion of their account. All sources used in this paper are properly cited.

Statistical Treatment of the Data

The **weighted mean** is used to determine the respondents' overall evaluation of each statement and each ISO 9126 characteristic. It is computed as:

$$WM = \Sigma(f \times w) / N$$

where *WM* is the weighted mean, *f* is the number of respondents who chose a rating, *w* is the weight of the rating (1 to 5), and *N* is the total number of respondents. The result is interpreted using the ranges in Table 3. **Frequency and percentage** are used to describe the profile of the respondents:

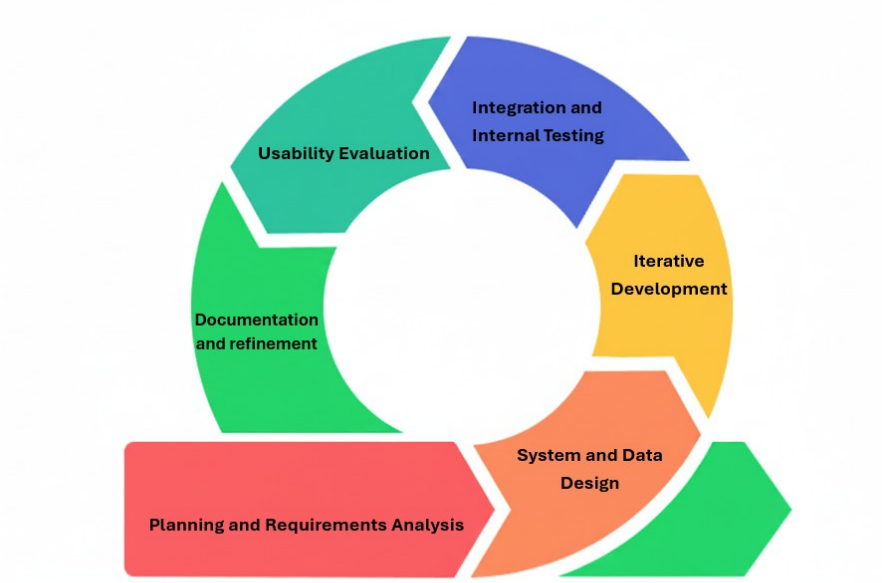
$$\% = (f / N) \times 100$$

where *f* is the frequency of a category and *N* is the total number of respondents. A sample computation is shown in Appendix F.

Software Development Methodology

To develop COBRA, the researchers selected the **Agile methodology** (Beck et al., 2001) because the system needed supportive, interactive, and flexible development. Features such as assessments, exercises, the coding sandbox, automated feedback, and learning analytics were built and refined in short cycles throughout the project. Figure 3 shows the phases.

Figure 3
Agile Methodology



Planning Phase

The planning phase involves gathering and analyzing the system requirements, objectives, and priorities to build a clear and secure foundation for the learning platform. During this phase, the researchers determined the required features, prioritized modules such as user accounts, lessons, exercises, feedback, and analytics, and compared existing platforms to outline the features COBRA needed.

Design Phase

The design phase focuses on the system's visual layout, color scheme, architecture, and workflows. In this phase, the researchers designed the learner and staff interfaces in Figma, the data flow and database structure, and how the Python lessons, exercises, activities, and tutorials would work.

Development Phase

The development phase is where the planned features, modules, and database structures are programmed. The researchers wrote the source code in Python (Flask), JavaScript, HTML, CSS, and SQL to build the lessons, the coding sandbox, the game-based activities, and the content management tools for mentors and administrators. Source code was managed in a Git repository on GitHub.

Testing Phase

The testing phase checks whether the developed features work correctly and meet the requirements. In this phase, the researchers performed functional and usability testing, including testing on desktop computers and on mobile devices, to verify the quality of the system.

Deployment Phase

The deployment phase makes the completed web-based system accessible to users over the internet. In this phase, the researchers deployed the Flask application and database to a server, applied the database migrations, and set up scheduled database backups.

Review and Feedback Phase

The review and feedback phase evaluates the deployed system and identifies improvements based on user testing. In this phase, the researchers use the respondents' evaluation and other feedback to improve the system's features.

Launch Phase

The launch phase is when the completed system is ready for its intended users after development, testing, review, and deployment. In this phase, the researchers make COBRA available to beginner Python learners and assess its usability, effectiveness, and learner satisfaction.

Calendar of Activities

Description of Activities

Table 4 describes the activities carried out in each phase of the project.

Table 4
Description of Activities

Phase	Activities	Output
Planning	Identify the problem and objectives; review related literature; compare existing platforms; list and prioritize requirements	Approved title and proposal; list of requirements
Design	Design learner and staff interfaces in Figma; design the database, system architecture, and course structure	UI prototypes; database design; architecture
Development	Code the eleven modules; prepare lesson content, videos, activities, and exercises	Working COBRA system
Testing	Functional testing of every module; usability testing on desktop and mobile; bug fixing	Tested system; list of fixes
Deployment	Set up the server, database, migrations, and backups	System accessible online
Review and Feedback	Validate the questionnaire; conduct the evaluation; analyze results; revise the system	Evaluation results; improved system
Launch	Release the system to learners; final defense and documentation	Final system and paper

Gantt Chart of Activities

Table 5 shows the schedule of activities. Please confirm the months against your actual timeline.

Table 5
Gantt Chart of Activities

Activity	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Planning							
Design							
Development							
Testing							
Deployment							
Review and Feedback							
Launch							

Resources

Hardware Requirements

Table 6 lists the hardware used to develop and test COBRA.

Table 6
Hardware Requirements

Hardware	Description/Use
 **Desktop Computer / Laptop**	A computer used for general computing, software development, and system testing; the primary device for developing COBRA.
 **Mobile Devices (Smartphones, Tablets)**	Portable touchscreen devices with internet access, used to test the responsiveness and availability of COBRA on smaller screens.

Hardware	Description/Use
 **Router**	A networking device that connects devices to a network, used to provide connectivity for development and testing and access to the web-based system.
 **Printer**	A device that produces paper copies of digital files, used to print the research paper, testing documents, and other project documentation.

Software Requirements

Table 7 lists the software and technologies used to develop COBRA.

Table 7
Software Requirements

Software	Description/Use
HTML	The markup language used to structure the content of the system's pages.
CSS	The stylesheet language used to control the presentation, layout, colors, and responsive design of the COBRA pages.
JavaScript	The programming language used for the interactive features of the system, including the game-based activities, the code editor, and the 3D scenes built with Three.js.

Software	Description/Use
 **Python**	The programming language used for the back end of COBRA through the Flask web framework; it handles the system logic and communication between the web pages and the database. Python is also the language taught by the system and is run in the browser through Pyodide.
 **Visual Studio Code**	A free source-code editor used for writing, editing, testing, debugging, and managing the project files.
 **Structured Query Language**	The language used to create, access, manage, and manipulate data in the relational database.
 **MySQL / MariaDB**	The relational database management system that stores COBRA's structured data such as accounts, course content, answers, progress, and logs.
 **XAMPP**	A local development environment that provides the MariaDB/MySQL server and phpMyAdmin used for developing and testing COBRA.
 **GitHub**	A cloud-based platform for hosting the source-code repository and tracking changes, allowing the researchers to work on the code together.
 **Figma**	A design tool used to create the interfaces, wireframes, and prototypes of the system; it allows several people to design at the same time.

Software	Description/Use
 **Google Forms**	A web-based tool for creating forms and collecting responses, used to gather the respondents' evaluation of the system.
Flask	A lightweight Python web framework used to build COBRA's pages, routes, and APIs.
Pyodide	Python compiled to WebAssembly; runs learners' code inside a Web Worker in the browser so lessons, exercises, and the sandbox work without any installation.
Three.js	A JavaScript 3D library used for the animated scenes of the learning activities.

Chapter 3

RESULT, ANALYSIS, AND DISCUSSION

This chapter presents the results of the study according to the specific problems stated in Chapter 1. It describes how each of the eleven modules of COBRA was developed, the respondents' evaluation of the system based on ISO 9126, and the system architecture of COBRA.

Research Problem 1: How to create a module for security?

Security Module

The security module was built on one shared set of sign-in rules used by both the learner login page and the separate staff login page for mentors and administrators. Passwords are never stored as plain text; they are hashed using the Werkzeug security library. A password must be at least eight characters long and include an uppercase letter, a lowercase letter, a number, and a special character. After five failed sign-in attempts, the account is locked for one minute, and every attempt and lockout is saved in the login and lockout logs.

Each login page admits only its own roles, and both answer with the same generic message ("Invalid username or password") so that the pages never reveal whether an account exists or what role it has. Forgotten passwords and usernames are recovered with a six-digit one-time code sent by email that expires after five minutes, and a new password can be set only after the code is verified. Every staff

page is protected by a role-permission map that denies access by default, so administrators handle accounts, monitoring, and reports, while mentors handle content and publishing. Learners are automatically logged out after 60 minutes without activity and staff after 120 minutes, with a warning 60 seconds before. Staff actions such as publishing, archiving, and password resets are written to an activity log.

Research Problem 2: How to create a module for user registration?

User Registration Module

Learners create their own accounts on the sign-up page by entering their personal details, email address, username, and password. The email address is verified with a six-digit one-time code before the account is created. Learners must be between 13 and 60 years old, and those under 18 must also declare that a parent or guardian agrees. Before using the system, every learner must accept the current Terms and Conditions and Privacy Notice; if either document is updated, learners are asked to accept it again.

Each account receives a unique, date-based account ID with a prefix for its role (for example, *LR2610090001* for a learner). Mentor and administrator accounts cannot be created through sign-up; they are created by an administrator. A new learner is welcomed with a notification and a first-time guided tour of the system. Learners can later edit their profile and photo, change their password, and

request the deletion of their account, which is archived after a 30-day period during which it can still be restored.

Research Problem 3: How to create a module for lesson management?

Lesson Management Module

The course is organized in four levels: **chapters**, **modules**, **lessons**, and each lesson's **activities and coding exercise**. The current course follows the topics of a beginner Python course: Getting Started, Variables and Simple Data Types, Introducing Lists, Working with Lists, If Statements, Dictionaries, User Input and While Loops, Functions, and Classes (Matthes, 2023).

Mentors manage this structure on the Manage Course page, where they can create, edit, reorder, archive, and restore chapters and modules, and on the Learning Resources page for lessons. Content is first saved as a draft and becomes visible to learners only after it is published on the Publishing page, which shows the whole course tree with each item's status and offers a preview of what learners will see. Changes to titles are recorded in a title history that can be reverted. A mentor can also mark a chapter as skippable (for example, "Getting Started") for learners who already know it.

On the learner side, the course appears as a visual **learning map**. Lessons within a module are taken in order, and the next module unlocks only when the learner finishes every lesson and reaches the **80% pass mark**. A lesson's grade is

computed as 50% from its activities and coding exercise and 50% from content progress (reading the content and watching the video).

Research Problem 4: How to create a module for learning resources?

Learning Resources Module

Each lesson's learning resources consist of lesson content and an optional video tutorial. Mentors write lesson content in a rich-text editor that supports headings, formatting, images, and Python code blocks. Code blocks in a lesson can be run by the learner directly on the page, so examples can be tried immediately. Headings also divide the lesson into parts, which are later used by the lesson recommendation module.

Video tutorials are added by pasting a YouTube link; the system stores only the video ID and embeds the video in the lesson page. The system records when a learner has read the lesson content and watched the video, and these count toward the lesson's content progress.

Research Problem 5: How to create a module for coding exercises?

Coding Exercises Module

A lesson may end with a coding exercise. Mentors create an exercise with instructions, an optional given input, the expected output, and optional "required in the code" elements (for example, that the solution must use a *for* loop or a function). When the learner submits, the code is run in the browser using Pyodide,

and the server marks the attempt correct only when (1) the code ran without a Python error, (2) its output matches the expected output, and (3) every required element is found in the code. The required elements are checked by parsing the code with Python's *ast* module; the code is never run on the server.

Feedback is immediate. When the attempt is wrong, the learner is told why – for example, which line caused an error and what the error means, how the output differs from what was expected, or which required element is missing. The number of attempts is recorded and shown in the learner's results.

Research Problem 6: How to create a module for learning activities?

Learning Activities Module

Each lesson has game-based learning activities of three types, each designed to measure a different skill (Table 8). Mentors build a pool of up to 50 items per activity, and each play draws five items the learner has not seen yet, so retakes show different questions. The points of an activity are always computed by the server from the actual number of items.

Table 8
Learning Activity Types and the Skills They Measure

Activity Type	Skill Measured	How It Works
Multiple Choice	Concepts and definitions	The learner chooses the correct answer from the options.
Fill in the Blanks	Code structure	The learner types the missing part of a line of code.
Flashcards	Reading code output	The learner gives what a piece of code prints; close answers are accepted.
Coding Exercise	Applying the lesson	The learner writes a complete program (see the Coding Exercises Module).

Activities are played in an animated 3D scene with lives and a question timer. A wrong answer, a skipped question, a timeout, or leaving the page costs a life. Every answer is graded at once and followed by feedback, and only the first-try score counts toward the lesson grade. When a module average is below the pass mark, the learner retakes only the items they missed until they pass.

Research Problem 7: How to create a module for a coding sandbox?

Coding Sandbox Module

The coding sandbox is a free-practice page with a code editor and an output console. Python runs in the learner's browser through Pyodide inside a Web Worker, so the page never freezes, even for an endless loop, and the server never runs the learner's code. The sandbox supports *input()*: the program pauses and waits for the learner to type in the console, just like a real terminal. For safety, a program is stopped after 15 seconds or 1,000 lines of output, and the learner can stop it at any time.

Learners can save their programs as snippets and open them later. Every run is logged with its status (success or error), which administrators can review on the Coding Sandbox monitoring page and which counts toward badges such as "Code Runner."

Research Problem 8: How to create a module for progress monitoring?

Progress Monitoring Module

Learners see their progress on the learning map, on each lesson's summary, and on their profile, which shows completed lessons, modules, and chapters, their scores, and their study time. On the staff side, the Learner Progress page shows every learner's average score and completion, by learner and by lesson. Scores are colored green (at or above the pass mark), yellow (within 10 points below it), or red.

Each learner is also given a status: *Not Started*, *On Track*, *Finished*, *Inactive* (no activity for seven days or more), or *Needs Help* (average below the pass mark, two or more retake rounds, or leaving activities three or more times). This lets mentors and administrators quickly find learners who need assistance. Administrators can also see who is online and what learners are currently doing.

Research Problem 9: How to create a module for learning analytics?

Learning Analytics Module

The learning analytics module turns learners' results into information. For each lesson, it computes the learner's **strengths** and **weaknesses** by skill from their first-try results: a skill is "Strong" at 80% or higher and "Needs work" below that, and for every weak skill the lesson parts behind the missed items are named. This works automatically, with no outside service, so the same results always produce the same explanation.

For administrators, the Analytics dashboard shows the total, active, and inactive learners, average scores, course completion, engagement over time, estimated study time, the most-missed questions, and the topics learners struggle with, filtered by period (7, 30, or 90 days, or all time) and compared with the previous period. The Reports page ranks learners by average score and completion and labels their performance as Excellent (90–100), Good (80–89), or Needs Improvement (below 80). Reports can be exported.

Research Problem 10: How to create a module for lesson recommendation?

Lesson Recommendation Module

When a learner misses an item, COBRA recommends the exact part of the lesson that teaches it. Each published lesson is split into parts at its headings. Key terms are taken from the missed item – mainly its correct answer, and also its question – while very common words are ignored and Python operators (such as ==, % and //) are kept. Each lesson part is scored by how many of these terms it contains, giving more weight to rarer terms and to terms in the part's heading. The highest-scoring part of the item's own lesson is recommended; earlier lessons are considered only when no part of the item's own lesson matches.

After finishing every lesson in a module, the learner sees a **Module Review** card that lists each missed item, the learner's answer and the feedback given, and the recommended lesson part to re-read. If the module is below the pass mark, the review and a retake of the missed items are required before the next module

unlocks. Mentors can follow every recommendation on the Recommendations page with its status: *Pending*, *In Progress*, or *Completed* (all missed items behind it have been fixed).

Research Problem 11: How to create a module for achievement tracking?

Achievement Tracking Module

Learners earn **badges** for reaching goals. Each badge has a rule made of a requirement type (for example, lessons completed, modules passed, or sandbox runs) and a required value. Badges are checked automatically, and a newly earned badge is saved with the date it was earned, shown on the learner's profile, and announced through the notification bell. A badge, once earned, is never taken back. Mentors can create, edit, and archive badges with their own icons. The system starts with the twelve badges in Table 9.

Table 9
Starter Badges of COBRA

Badge	Requirement
First Steps	Complete your first lesson.
Lesson Learner	Complete 10 lessons.
Module Master	Pass your first module.
Chapter Champion	Complete a whole chapter.
Quiz Ace	Perfect first-try score on a Multiple Choice activity.
Blank Buster	Perfect first-try score on a Fill in the Blanks activity.
Card Shark	Perfect first-try score on a Flashcards activity.
Problem Solver	Pass a coding exercise.
Code Runner	Run code in the Sandbox 25 times.
Snippet Saver	Save 5 snippets in the Sandbox.
Dedicated	Reach 10 hours of learning time.
Consistent	Log in on 7 different days.

When a learner passes every published chapter (skipped chapters excluded), the system issues a **Certificate of Completion** with a unique reference number. It is issued only once, its completion date never changes, and it always shows the learner's current name.

Research Problem 12: How do the respondents evaluate the COBRA system with respect to the ISO 9126 metrics on functionality, reliability, usability, and efficiency?

Evaluation of the System Based on ISO 9126

[To be completed after the evaluation. Fill in the weighted mean (WM) and verbal interpretation (VI) of each statement, then discuss the results below each table.]

Table 10
Respondents' Evaluation of COBRA in Terms of Functionality

Indicator	WM	VI
1. The system provides all the functions needed to learn basic Python programming.		
2. The lessons, activities, and exercises give correct results and feedback.		
3. The coding sandbox runs Python programs correctly.		
4. The system protects accounts and data from unauthorized access.		
5. The system correctly records progress, scores, badges, and recommendations.		
Overall Weighted Mean		

[Discussion of the results for functionality.]

Table 11
Respondents' Evaluation of COBRA in Terms of Reliability

Indicator	WM	VI
1. The system works without errors or crashes during use.		
2. The system handles wrong inputs and errors without losing data.		
3. The system saves progress so a learner can continue where they left off.		
4. The system is available whenever I need to use it.		
Overall Weighted Mean		

[Discussion of the results for reliability.]

Table 12
Respondents' Evaluation of COBRA in Terms of Usability

Indicator	WM	VI
1. The system is easy to understand and use.		
2. The pages, buttons, and menus are clear and well organized.		
3. Learning how to use the system takes little effort.		
4. The design of the system is attractive and engaging.		
5. The system can be used comfortably on both computers and mobile devices.		
Overall Weighted Mean		

[Discussion of the results for usability.]

Table 13
Respondents' Evaluation of COBRA in Terms of Efficiency

Indicator	WM	VI
1. The pages load quickly.		
2. Code runs and results appear within an acceptable time.		
3. The system responds quickly to my actions.		
4. The system does not slow down my device.		
Overall Weighted Mean		

[Discussion of the results for efficiency.]

Table 14
Summary of the Respondents' Evaluation of COBRA

ISO 9126 Characteristic	WM	VI
Functionality		
Reliability		
Usability		
Efficiency		
Overall Weighted Mean		

[Overall discussion of the evaluation results.]

Research Problem 13: What system architecture was proposed for the COBRA system?

System Architecture

COBRA uses a three-tier web architecture, shown in Figure 4.

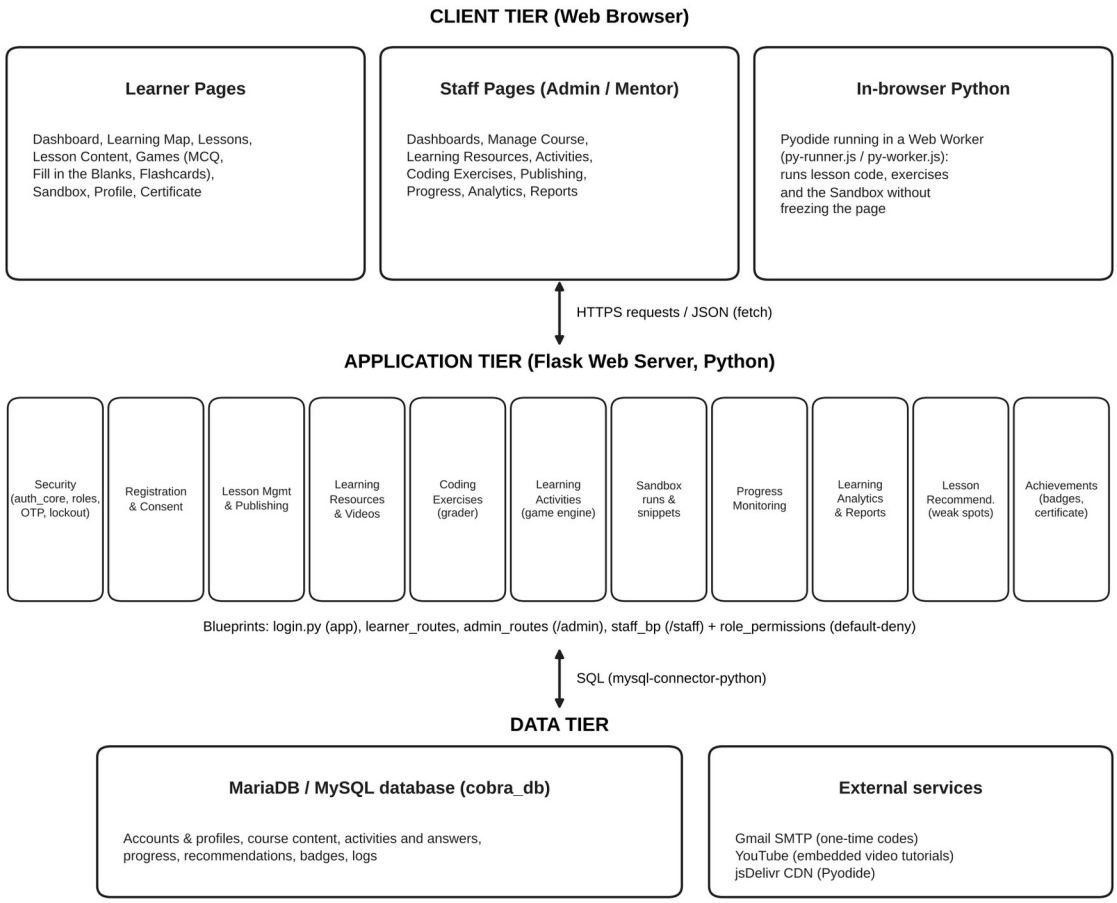
The **client tier** is the web browser. It displays the learner pages (dashboard, learning map, lessons, activities, sandbox, profile, and certificate) and the staff pages (dashboards, course and content management, publishing, progress, analytics, and reports). Python code from lessons, exercises, and the sandbox runs in the browser using Pyodide inside a Web Worker, which keeps the page responsive and keeps learners' code off the server.

The **application tier** is a Flask web server written in Python. It is organized into blueprints for the learner pages, the staff pages under */admin*, the staff login under */staff*, and shared services such as consent, notifications, and idle logout. Each of the eleven modules is implemented as its own set of Python files, and every staff request is checked against a default-deny role-permission map.

The **data tier** is a MariaDB/MySQL database (*cobra_db*) that stores accounts and profiles, the course structure and content, activity items and answers, progress, recommendations, badges, certificates, and logs. The system also uses Gmail's SMTP server to send one-time codes, YouTube to host the video tutorials, and a content delivery network to load Pyodide.

Figure 4

System Architecture of COBRA



Chapter 4

SUMMARY OF FINDINGS, CONCLUSIONS, AND RECOMMENDATIONS

This chapter presents the summary of findings, the conclusions drawn from them, and the recommendations of the researchers.

Summary of Findings

Relative to Problems 1–2:

Security and user registration were achieved through hashed passwords with strength rules, a five-attempt lockout, email one-time codes, separate learner and staff sign-in pages, a default-deny role-permission map, idle logout, activity logs, verified learner sign-up with age checking and consent, and administrator-created staff accounts.

Relative to Problems 3–4:

Lessons are managed as chapters, modules, and lessons with a draft-and-publish workflow, previews, ordering, archiving, and skippable chapters. Lessons combine rich content with runnable code and embedded video tutorials, and modules unlock in order at an 80% pass mark.

Relative to Problems 5–7:

Coding exercises are graded immediately on errors, output, and required code elements; learning activities offer three game types with item pools, lives,

timers, feedback, and retakes; and the coding sandbox runs Python in the browser with input support, safety limits, saved snippets, and run monitoring.

Relative to Problems 8–9:

Progress is monitored by learner and by lesson with colored score bands and learner statuses, while learning analytics identify per-lesson strengths and weaknesses by skill and provide dashboards and performance reports.

Relative to Problems 10–11:

Every missed item is matched to the lesson part that teaches it and gathered in a Module Review that mentors can follow, and learners are recognized through mentor-managed badges, notifications, and a certificate of completion.

Relative to Problem 12:

[Summarize the weighted mean and interpretation for functionality, reliability, usability, efficiency, and overall.]

Relative to Problem 13:

A three-tier architecture was used: a browser client with in-browser Python (Pyodide), a Flask application server, and a MariaDB/MySQL database.

Conclusions

Relative to Findings 1–2:

COBRA protects its users and data with layered security and lets beginners register safely and responsibly, while keeping staff accounts under administrator control.

Relative to Findings 3–4:

A structured, ordered course with short lessons, runnable examples, and videos can guide beginners step by step and make sure they master each module before moving on.

Relative to Findings 5–7:

Immediate, specific feedback in exercises and game-based activities, together with a safe place to experiment, gives beginners frequent and engaging practice.

Relative to Findings 8–9:

Automatically computed progress, statuses, strengths, and weaknesses allow both learners and staff to see where help is needed without manual checking.

Relative to Findings 10–11:

Linking each mistake to the exact lesson part that teaches it personalizes review, and badges and a certificate give learners goals that keep them motivated.

Relative to Finding 12:

[State whether COBRA met the expected rating in each ISO 9126 characteristic.]

Relative to Finding 13:

The three-tier architecture with in-browser Python allows the system to run code safely and responsively without installing Python on the learner's device.

Recommendations

Relative to Conclusions 1–2:

Move all secrets (such as the email password and session key) into environment variables on the server, add optional two-factor authentication for staff, and periodically review the activity and login logs.

Relative to Conclusions 3–4:

Add more chapters beyond the beginner topics (e.g., files, exceptions, and testing) and allow downloadable materials for learners with limited connectivity.

Relative to Conclusions 5–7:

Add more exercises per lesson with progressive hints, and allow the sandbox to work with multiple files.

Relative to Conclusions 8–9:

Allow mentors to send messages or reminders directly to learners marked “Needs Help” or “Inactive.”

Relative to Conclusions 10–11:

Extend recommendations to suggest extra practice items for weak skills, and add leaderboards or class groups to encourage friendly competition.

Relative to Conclusion 12:

[Recommendations based on the lowest-rated ISO 9126 indicators.]

Other Recommendations

1. Future researchers may develop a native mobile application or offline mode for COBRA.

2. Future researchers may add SMS notifications for learners.
3. Future researchers may extend COBRA to other programming languages.
4. Future researchers may conduct a longer study comparing the performance of learners who use COBRA with those who do not.
5. The school may adopt COBRA as a supplementary tool for its introductory programming courses.

REFERENCES

Books

Bloom, B. S. (1968). Learning for mastery. *Evaluation Comment, 1*(2), 1–12. UCLA Center for the Study of Evaluation of Instructional Programs.

Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3rd ed.). No Starch Press.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Published Theses and Dissertation

[Add local or foreign theses related to e-learning or programming education systems, if any.]

Journal and Articles

Alasmari, O., Singer, J., & Bikanga Ada, M. (2024). Do current online coding tutorial systems address novice programmer difficulties? In *Proceedings of the 15th International Conference on Education Technology and Computers (ICETC '23)* (pp. 242–248). ACM. https://doi.org/10.1145/3629296.3629333

Chen, Y., et al. (2024). [Complete citation needed – title, journal, volume, pages, DOI.]

Iftikhar, S. (2022). [Complete citation needed – title, journal, volume, pages, DOI.]

Likert, R. (1932). A technique for the measurement of attitudes. *Archives of Psychology, 22*(140), 1–55.

Marwan, S., Akram, B., Barnes, T., & Price, T. W. (2022). Adaptive immediate feedback for block-based programming: Design and evaluation. *IEEE Transactions on Learning Technologies, 15*(3), 406–420. https://doi.org/10.1109/TLT.2022.3180984

Menon, P. (2023). [Complete citation needed – title, journal, volume, pages, DOI.]

Oka, H., Ohnishi, A., Nishida, T., Terada, T., & Tsukamoto, M. (2024). A choice-based programming learning method to develop problem-solving skills. *IEEE Access, 12*, 119550–119562. https://doi.org/10.1109/ACCESS.2024.3443879

Ryan, R. M., & Deci, E. L. (2000). Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *American Psychologist, 55*(1), 68–78. https://doi.org/10.1037/0003-066X.55.1.68

Samosa, R. C. (2021). *From test validity to construct validity & back: Theory and application in the experimental design in science action research*. Philippine Association of Physics and Science Instructors, De La Salle University.

Skulmowski, A., & Xu, K. M. (2022). Understanding cognitive load in digital and online learning: A new perspective on extraneous cognitive load. *Educational Psychology Review, 34*(1), 171–196. https://doi.org/10.1007/s10648-021-09624-7

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science, 12*(2), 257–285. https://doi.org/10.1207/s15516709cog1202_4

Uhanova, M., Prokofjeva, N., Kataļņikova, S., Zavjalova, O., & Ziborova, V. (2023). The influence of prior knowledge and additional courses on the academic performance of students in the introductory programming course CS1. *Procedia Computer Science, 225*, 1397–1406. https://doi.org/10.1016/j.procs.2023.10.128

Wu, S., et al. (2024). [Complete citation needed – title, journal, volume, pages, DOI.]

Online Sources

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., … Thomas, D. (2001). *Manifesto for agile software development*. https://agilemanifesto.org/

GitHub. (2024, October 29). *Octoverse: AI leads Python to top language as the number of global developers surges*. The GitHub Blog. https://github.blog/news-insights/octoverse/octoverse-2024/

International Organization for Standardization. (2001). *Software engineering – Product quality – Part 1: Quality model* (ISO/IEC Standard No. 9126-1:2001). https://www.iso.org/standard/22749.html

Pallets. (n.d.). *Flask documentation*. https://flask.palletsprojects.com/

Pyodide. (n.d.). *Pyodide documentation*. https://pyodide.org/

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Official Gazette of the Republic of the Philippines. https://www.officialgazette.gov.ph/2012/08/15/republic-act-no-10173/

APPENDIX A:
LETTER TO RESPONDENTS

COLEGIO DE SAN GABRIEL ARCANGEL, INC.
College of Computer Studies
City of San Jose del Monte, Bulacan

Date: _____

Sir / Madam:

Greetings of Peace and Joy!

We are the students of Colegio de San Gabriel Arcangel, Inc. conducting a study entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION,"** which aims to develop a web-based learning system that helps beginners learn Python programming. This study is part of the requirements for the degree of Bachelor of Science in Information Technology.

In this regard, we would like to request your participation in evaluating the system by using it and answering our survey questionnaire. Rest assured that the data gathered will be held with the utmost confidentiality in accordance with the Data Privacy Act of 2012.

Your favorable response will be of great help to the realization of this study.

Thank you very much.

Very truly yours,

Name of Student Lead Researcher
Student Lead Researcher

Noted by:

Prof. Jimmy D.V. Roldan	**Jimmy D.V. Roldan, MSIT**
Research Group's Adviser	Research Course Facilitator

APPENDIX B:
LETTER OF PERMISSION TO THE DEAN

COLEGIO DE SAN GABRIEL ARCANGEL, INC.
College of Computer Studies
City of San Jose del Monte, Bulacan

Date: _____

Name of Dean
Dean, College of Computer Studies
Colegio de San Gabriel Arcangel, Inc.

Sir / Madam:

Greetings of Peace and Joy!

We are the students of Colegio de San Gabriel Arcangel, Inc. conducting a study entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION,"** which aims to develop a web-based learning system that helps beginners learn Python programming. This study is part of the requirements for the degree of Bachelor of Science in Information Technology.

In this regard, we would like to request your good office **to allow us to conduct the evaluation of our system with the students and faculty of the college**. Rest assured that the data gathered will be held with the utmost confidentiality.

Your favorable response will be of great help to the realization of this study.

Thank you very much.

Very truly yours,

Name of Student Lead Researcher
Student Lead Researcher

Noted by:

Prof. Jimmy D.V. Roldan	**Jimmy D.V. Roldan, MSIT**
Research Group's Adviser	Research Course Facilitator

APPENDIX C:
LETTER TO EXPERT VALIDATORS
FOR THE VALIDATION OF THE SURVEY QUESTIONNAIRE

COLEGIO DE SAN GABRIEL ARCANGEL, INC.
College of Computer Studies
City of San Jose del Monte, Bulacan

Date: _____

Sir / Madam:

Greetings of Peace and Joy!

We are the students of Colegio de San Gabriel Arcangel, Inc. conducting a study entitled **"COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION."** This study is part of the requirements for the degree of Bachelor of Science in Information Technology.

In this regard, may we have the pride and honor of having you as one of the research experts to validate our survey questionnaire by giving your invaluable comments and suggestions. Your much-sought participation will lend the necessary credence to our research work.

Your favorable attention and action on our request will significantly help the complete realization of this study. Thank you very much.

Very truly yours,

Name of Student Lead Researcher
Student Lead Researcher

Noted by:

Prof. Jimmy D.V. Roldan	**Jimmy D.V. Roldan, MSIT**
Research Group's Adviser	Research Course Facilitator

APPENDIX D:
RESEARCH INSTRUMENT VALIDATION SHEET
FOR QUANTITATIVE RESEARCH

Name of Validator: _____
Degree/Specialization: _____
Position/Designation: _____

Research Title: COBRA: COMPREHENSIVE ONLINE BEGINNER RESOURCE FOR PYTHON APPLICATION

Direction: This tool asks for your evaluation of the questionnaire to be used in data gathering for the investigation stated above to establish its validity. You are requested to give your honest assessment using the criteria stated below. Please choose only one from the selection.

Scale	Interpretation	Description
4	Highly Valid	The questionnaire is highly valid and can provide unbiased data for investigation, allowing 0–5% error.
3	Valid	The questionnaire is valid and can provide unbiased data for investigation, allowing 6–10% error.
2	Less Valid	The questionnaire is less valid, unacceptable, and needs revisions.
1	Not Valid at All	The questionnaire is not valid at all, unacceptable, and needs revisions.

Indicators	4	3	2	1
Clarity and Direction of Items				
1. The vocabulary level, language, and structure are at the conceptual level of the participants/respondents.	☐	☐	☐	☐
2. The test directions and the items are written clearly and understandably.	☐	☐	☐	☐
Presentation and Organization of Items				
3. The items are presented and organized in a logical manner.	☐	☐	☐	☐
Suitability of Items				
4. The items appropriately present the substance/content of the research.	☐	☐	☐	☐
5. The questions/items are designed to determine what is supposed to be measured.	☐	☐	☐	☐
Adequateness of the Content				
6. The number of questions/items per domain/area is representative enough of all the questions needed for the research.	☐	☐	☐	☐
Attainment of the Purpose				
7. The instrument as a whole can fulfill the objectives needed for the research.	☐	☐	☐	☐

Indicators	4	3	2	1
Objective				
8. Each item question/indicator requires only one specific answer or measures only one behavior.	☐	☐	☐	☐
Scale and Evaluation of Rating				
9. The scale adapted/used is appropriate for the item/indicator.	☐	☐	☐	☐

Remarks: *(recommendations for the improvement of the tool)*

_____

_____

_____

_____

_____

_____

Evaluator's Signature Over Printed Name

Adapted from:

Samosa, R. C. (2021). From Test Validity to Construct Validity & Back: Theory and Application in the Experimental Design in Science Action Research. Philippine Association of Physics and Science Instructors, De La Salle University, Manila.

APPENDIX E:
SURVEY QUESTIONNAIRE

Part I. Respondent's Profile

Name (optional): _____

Respondent type: ☐ Beginner learner ☐ IT instructor ☐ IT expert

Prior Python experience: ☐ None ☐ Less than 6 months ☐ 6 months or more

Part II. Evaluation of COBRA (ISO 9126)

Direction: Please rate each statement by checking the box that best describes your evaluation of the system. 5 – Strongly Agree, 4 – Agree, 3 – Neutral, 2 – Disagree, 1 – Strongly Disagree.

Indicators	5	4	3	2	1
FUNCTIONALITY					
1. The system provides all the functions needed to learn basic Python programming.	☐	☐	☐	☐	☐
2. The lessons, activities, and exercises give correct results and feedback.	☐	☐	☐	☐	☐
3. The coding sandbox runs Python programs correctly.	☐	☐	☐	☐	☐
4. The system protects accounts and data from unauthorized access.	☐	☐	☐	☐	☐
5. The system correctly records progress, scores, badges, and recommendations.	☐	☐	☐	☐	☐
RELIABILITY					
1. The system works without errors or crashes during use.	☐	☐	☐	☐	☐
2. The system handles wrong inputs and errors without losing data.	☐	☐	☐	☐	☐
3. The system saves progress so a learner can continue where they left off.	☐	☐	☐	☐	☐
4. The system is available whenever I need to use it.	☐	☐	☐	☐	☐
USABILITY					
1. The system is easy to understand and use.	☐	☐	☐	☐	☐
2. The pages, buttons, and menus are clear and well organized.	☐	☐	☐	☐	☐
3. Learning how to use the system takes little effort.	☐	☐	☐	☐	☐
4. The design of the system is attractive and engaging.	☐	☐	☐	☐	☐
5. The system can be used comfortably on both computers and mobile devices.	☐	☐	☐	☐	☐
EFFICIENCY					
1. The pages load quickly.	☐	☐	☐	☐	☐
2. Code runs and results appear within an acceptable time.	☐	☐	☐	☐	☐
3. The system responds quickly to my actions.	☐	☐	☐	☐	☐
4. The system does not slow down my device.	☐	☐	☐	☐	☐

Comments and suggestions:

_____

_____

_____

APPENDIX F:
FORMULA WITH SAMPLE COMPUTATION OF STATISTICS USED

Weighted Mean

$$WM = \Sigma(f \times w) / N$$

where WM = weighted mean, f = frequency of each rating, w = weight of the rating, N = total number of respondents.

Sample computation (illustrative only – replace with an actual item from your results). Suppose 40 respondents rated an item as follows: 20 chose 5, 15 chose 4, and 5 chose 3.

$WM = [(20 \times 5) + (15 \times 4) + (5 \times 3)] / 40 = (100 + 60 + 15) / 40 = 175 / 40 = 4.38$

Since 4.38 falls within 4.21 – 5.00, the verbal interpretation is **Strongly Agree**.

Percentage

$$\% = (f / N) \times 100$$

Example: if 30 of 40 respondents are beginner learners, $\% = (30 / 40) \times 100 = 75\%$.

APPENDIX G:
RAW DATA

[Insert the tabulated responses of the respondents from Google Forms.]

APPENDIX H:
SYSTEM SCREENSHOTS

[Insert labeled screenshots of the main pages: Landing Page, Sign Up, Login, Learning Map, Lesson Content, Learning Activities, Coding Exercise, Coding Sandbox, Module Review, Profile and Badges, Certificate, Mentor Manage Course and Publishing, Admin Dashboard, Learner Progress, Analytics, and Reports.]

CURRICULUM VITAE

Insert formal picture here

MARK DANZEN Q. AQUINO
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Mark Danzen Q. Aquino
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

DARELL F. BACANI
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Darell F. Bacani
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

CHRISTIAN GOLD DC. ESPAÑOLA
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Christian Gold DC. Española
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

MARKGIL L. GAMBOA
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Markgil L. Gamboa
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

IRISH I. GREGORIO
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Irish I. Gregorio
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

NICOLE S. MIJOY
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Nicole S. Mijoy
Signature over printed name

CURRICULUM VITAE

Insert formal picture here

EMMANUEL A. SPICER
Address: [Address]
Contact No.: [Contact No.]
Email Address: [Email Address]

EDUCATIONAL BACKGROUND:

TERTIARY:
- Bachelor's Degree: Bachelor of Science in Information Technology
- Institution Name: Colegio de San Gabriel Arcangel, Inc.
- Expected Year of Graduation: ____

SENIOR HIGH SCHOOL:
- Track/Strand:
- Institution Name:
- Year Graduated:
- Honors / Awards:

JUNIOR HIGH SCHOOL:
- Institution Name:
- Year Graduated:
- Honors / Awards:

PRIMARY:
- Institution Name:
- Year Graduated:

WORK EXPERIENCE:

Company Name:
Designation:
Address:
Inclusive Dates:

PERSONAL INFORMATION:

Age:
Sex:
Date of Birth:
Place of Birth:
Civil Status:
Religion:

I hereby certify that all information stated above is true and correct.

Emmanuel A. Spicer
Signature over printed name